

Mathlib and Other Lean Contributions

Evelyn Sander



George Mason University

Lean Learning Seminar, Fairfax VA, October 7, 2026

Contributing Pull Requests (PR) to Mathlib

Decide what to contribute

- Small fixes and single lemmas are almost always welcome.
- Longer PRs that extend an existing theory are generally accepted.
- A new theory needs a scope discussion first: maintainers have to be able to review and maintain it. Therefore it must be related to their fields.
- Purely stylistic PRs may be closed without the file authors' approval.
- Consider the possibility of creating a standalone repository, adding Mathlib as a dependency. Lean repositories are also indexed.

Read first

[Contributing to Mathlib: guidelines](#)

Find gaps

[Mathlib overview](#)

[Undergrad coverage](#)

[Missing topics](#)

Check it isn't already there

[Loogle](#) declaration search

[Docs site](#): import hierarchy; each declaration links to its source

Indexed repositories

[Reservoir](#): the Lean package registry; browse existing Lean projects

Who reviews your area?

[Mathlib maintainers](#) approve changes; [reviewers](#) assist with PR review. All teams: [teams page](#)

2

maintainers whose bios say PhD
in dynamical systems

4

reviewers tagged t-dynamics

0

mentions of ODEs, PDEs, or
numerical analysis on either page

Gap: applied math, ODEs and numerics are not named in these team bios. One maintainer's bio mentions a PhD in applied mathematics. Bios are self-written; a missing keyword is not proof of missing expertise.

Talk to people first

- [Lean Zulip](#): introduce yourself, ask whether the idea fits, and look for related work.
- Announce larger projects so others don't duplicate them.

Write your own comments

Mathlib's AI policy: “Using an LLM when writing comments on GitHub or Zulip is not allowed: use your own words.” Repeated violations risk suspension or a ban from opening PRs and from Zulip. (Note the meta-ness of this comment, as this talk was mostly drafted by Claude directly after I had just given a strong warning to not go rogue and write a comment.)

The mechanics

- 1 Fork `mathlib4` and clone it.
- 2 Run `lake exe cache get` to download prebuilt oleans instead of compiling everything.
- 3 Create one branch per contribution.
- 4 If you add new files, run `lake exe mk_all`.
- 5 Open the PR against the main repo's master, following the commit and title conventions.

Details: [How to contribute](#) · [Git guide](#)

The mechanical workflow



The review loop repeats until a maintainer approves.

Style and naming

Guides: [style](#) · [naming](#)

- Lines are at most 100 characters, with 2-space indentation.
- Every file needs a header and a module docstring.
- Names follow fixed patterns: `snake_case` for theorems, `UpperCamelCase` for types, and the `C_of_A_of_B` pattern for conclusions and hypotheses (examples on the next slide).
- Changing or deleting a declaration requires a `@[deprecated]` attribute with a date.
- If you touch core files, comment `!bench` on the PR to run the benchmark.

Naming by example

What	Convention	Mathlib example
Types, structures, classes	UpperCamelCase	TopologicalSpace, Subgroup
Theorems and proofs	snake_case	lt_of_le_of_ne, birkhoffSum_succ
Functions and other terms	lowerCamelCase, named like their return value	birkhoffSum, Polynomial.newtonMap

Reading C_of_A_of_B

The conclusion comes first. Each hypothesis follows after of, in the order the hypotheses appear.

A theorem $A \rightarrow B \rightarrow C$ is named C_of_A_of_B.

Standard abbreviations: lt, le, ne, pos, nonneg, ...

Example

theorem lt_of_le_of_ne :

$$a \leq b \rightarrow a \neq b \rightarrow a < b$$

conclusion $a < b$ gives lt; the hypotheses $a \leq b$ and $a \neq b$ give le and ne, so lt_of_le_of_ne.

[Source](#) · [naming guide](#)

Review cycle: the key terms

CI (continuous integration)

Servers automatically build Mathlib with your PR as soon as it is opened. The PR page shows a green tick (passed), a yellow circle (running) or a red cross (failed). Reviewers look at PRs that build.

Reviewers and maintainers

Anyone can review. Reviewers are experienced members. Maintainers give final approval and are the ones who can merge.

bors and “bors merge”

bors is the bot that runs the merge queue. A maintainer approves by commenting bors merge; bors then builds the PR on top of current master and merges it only if that build succeeds.

Label: awaiting-author

A tag on the PR meaning the ball is in your court. A reviewer adds it with comments; you fix things, then remove it to send the PR back for review.

Label: maintainer-merge

Added by a reviewer who thinks the PR is ready. It tells maintainers to look at it next; it does not merge anything.

The merge queue

Labels show where you are: ready-to-merge (in the queue), bors-staging (being built now), awaiting-requeue (dropped after a failed build). Pushing a new commit takes the PR off the queue.

The general life cycle of a PR

It varies a lot.

Size

The contribution guide says smaller PRs get reviewed faster.

Familiarity

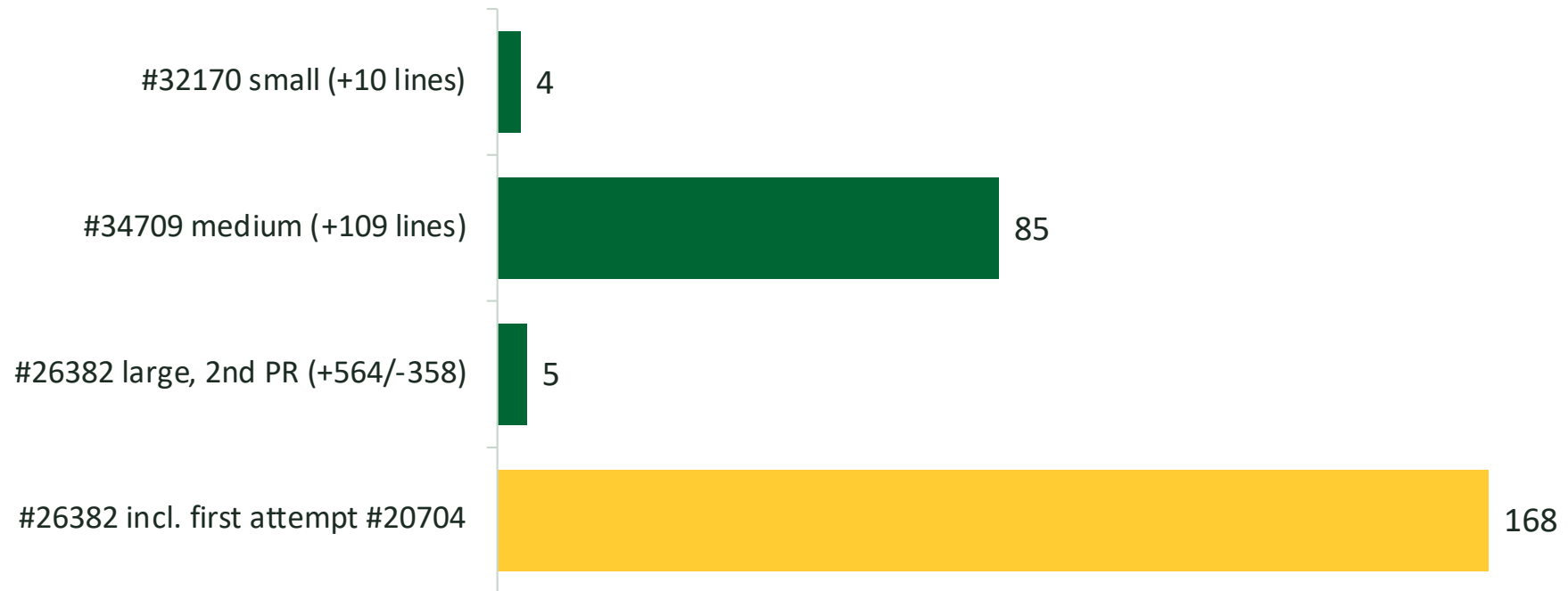
Review time depends on how much of the area reviewers know.

Your response time

...and on how quickly you respond to feedback.

- Mathlib has a review dashboard, [queueboard](#), which aims to give every open PR a timely response and tracks total time in review. I did not find a published median review time.
- After the reviewer cycle comes maintainer approval and the merge queue; each stage adds some delay.
- The guide's own expectation: if your PR builds, someone will review it within a few weeks, depending on its size.

Three real PRs: days from opening to merge



Units: calendar days from the day the PR was opened to the day bors merged it. Examples, not a distribution; dates from the public PR timelines on GitHub (UTC).

What happened in each

#32170

gronwallBound_mono

- 1 file, +10 lines; first-time contributor
- Same day: bot labels, reviewer suggestions, awaiting-author
- Author fixes; second review; ready-to-merge; merged 4 days after opening

#34709

discrete Grönwall inequality

- New file +95 lines, import list, bibliography entry
- Triage asked about LLM use; LLM-generated label added
- Merge conflict, two review rounds, one CI build retried; merged after 12 weeks

#26382

existence of local flows

- +564/-358 lines, 6 files, 30 commits
- Rewrites an earlier PR (#20704) and credits its author
- Approved by a reviewer, maintainer-merge label, then a maintainer's bors merge; merged 5 days after reopening

Upstream versus downstream

Upstream = into Mathlib

Your code lands in Mathlib itself. Everyone who imports Mathlib gets it, and Mathlib's reviewers and maintainers review and maintain it.

Right when:

- it is general and reusable
- it fits Mathlib's scope
- you can stay for the review rounds

Downstream = on top of Mathlib

Your own repository imports Mathlib as a dependency. You set the review rules and release schedule, and you maintain it.

Right when:

- it is research-specific or applied
- you need speed or exploration
- it is large or AI-generated at scale

[Reservoir](#) lists Lean projects. Mathlib even has PR labels for upstreaming from downstream projects such as FLT, carleson and sphere-packing.

Often both: the [Liquid Tensor Experiment](#) (formalizing a theorem of Clausen and Scholze on liquid real vector spaces; [completion post](#)) lived in its own repository, and parts were gradually moved into Mathlib. The radii polynomial framework (later slides) is downstream, with some pieces proposed for upstream.

6. AI and LLM policy

What Mathlib asks of you

- Understand everything an AI wrote, including design decisions, and justify each decision to reviewers without AI.
- Disclose in the PR description which tools you used and how.
- Add the LLM-generated label for substantial AI-generated code.
- No LLM-written comments on GitHub or Zulip.
- Low-quality LLM PRs are closed without comment; repeated violations risk a ban.

Alternatives to follow

- [TauCeti](#): AI-authored library with its own review rubrics (next slides)
- [Lean Pool](#): an AI-maintained archive of formalized math
- Your own downstream library, under your own rules

7. Alternative route: TauCeti

- A separate Apache-2.0 library built on top of Mathlib: not an upstream contribution path. [Site](#), [about](#), [statistics](#).
- AI agents write the Lean. A human-owned roadmap sets the targets, and merging requires every review rubric to approve the current commit.
- A useful contrast: it shows Mathlib's review standards (correctness, reuse, API design, naming, generality) written down as rubrics.
- The code will never be directly upstreamed, though it might provide inspiration for future Mathlib development.

Three repositories

- [TauCeti](#): the AI-authored Lean
- [TauCetiRoadmap](#): human-controlled targets
- [TauCetiReview](#): review rubrics and machinery

Similar projects: Lean Pool's related-work section compares TauCeti with the Archive of Formal Proofs and the Palomar Registry.

Pros and cons of Mathlib contributions

Pros	Cons
Your results are permanent, discoverable and citable, and others build on them	Review latency can be weeks to months, and reviewer bandwidth is the bottleneck
Expert feedback on API design improves your Lean	The style, naming and generality bar is high, so a quick proof can take much more work to land
Community and visibility, with a path to reviewer or maintainer roles	The maintenance burden is yours: refactors, deprecations and breaking changes
Free CI, caching, and benchmark infrastructure	It rewards general lemmas and not necessarily your research-specific results
Credit for your work through git history and a public track record	A slow merge queue and frequent upstream changes mean you must keep your branch up to date
Prompts you to formalize cleanly, which is good for a thesis or paper	The AI-use policy adds disclosure and understanding requirements

The main trade-off

Upstream

Gives longevity and quality, at the cost of time.

Downstream

A library of your own, or TauCeti-style: faster and freer, but less visible and less maintained.

Examples: dynamics and numerics

Birkhoff/Ergodic theory · Poincaré–Bendixson · Newton–Kantorovich · AMBER

Birkhoff: a lot already exists

In Mathlib (files on master)

[Dynamics/BirkhoffSum/](#)

Basic, Average, NormedSpace, QuasiMeasurePreserving

[Dynamics/Ergodic/](#)

Ergodic, MeasurePreserving, Conservative, Function, AddCircle, EmpiricalMeasure, RadonNikodym, Action/ ...

[Analysis/InnerProductSpace/MeanErgodic](#)

von Neumann mean ergodic theorem (Hilbert space)

Ongoing development

Examples: [#42078](#) (open) pointwise and [#34031](#) maximal ergodic theorem (open).

How to find this out

1. Browse the docs

Open the [docs site](#), find Mathlib.Dynamics in the module list (or use the search box), and read the module docstrings.

2. Search

[Loogle](#): the quoted word “birkhoff” matches 88 declaration names, e.g. birkhoffSum.

[GitHub PR search](#): is:pr birkhoff in:title lists open and closed PRs, so you see unfinished work.

Watch for name clashes: other “Birkhoff” results (the lattice representation theorem, Birkhoff–von Neumann for doubly stochastic matrices) appear too.

Poincaré–Bendixson: outside Mathlib

Status

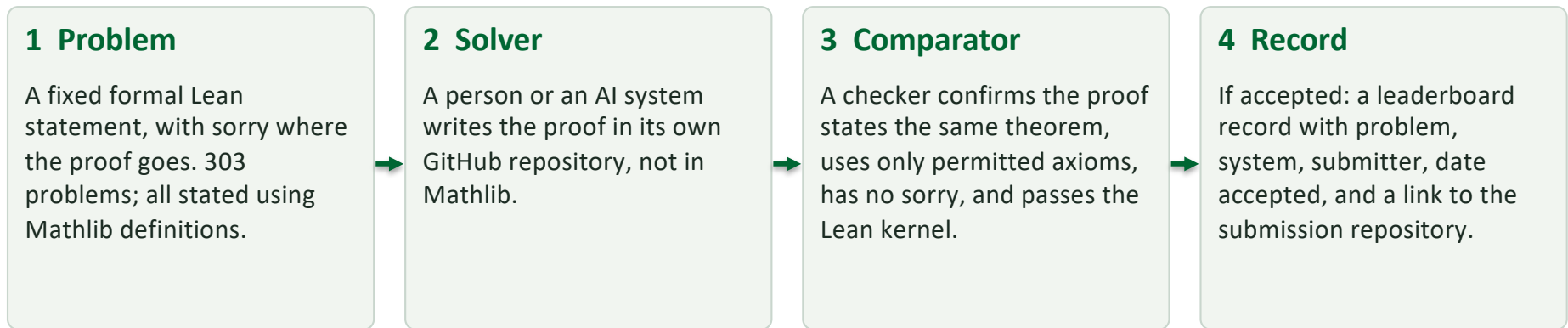
Isabelle/HOL (done): Immler and Tan, [CPP 2020](#); also in the [Archive of Formal Proofs](#). It builds on earlier Isabelle Picard–Lindelöf work and uses Harrison's Jordan curve theorem from HOL Light.

Lean: not in Mathlib. The [Lean eval page](#) lists the statement (with sorry), and its leaderboard records accepted solutions from five AI systems between 2026-07-01 and 2026-09-05. None was submitted to Mathlib. What these records are: next slide.

Why it is hard for Mathlib

- **Jordan curve theorem:** not in Mathlib per its [eval page](#); winding numbers and invariance of domain are also missing. A closed orbit splits the plane into inside and outside.
- **Global ODE theory:** needs solutions and flows behaving well on all of the plane.
- **Dynamics machinery:** periodic orbits and first-return maps on transverse sections.

What the Lean eval records are



System that solved it	Accepted (2026)
Aristotle (Harmonic)	July 1
Seed Prover (ByteDance)	July 12
Humanifa + GPT 5.6 sol	July 28
Axiom Prover (Axiom Math)	August 29
Tau (caj.al)	September 5

Not part of Mathlib. The proofs sit in the submitters' repositories, and the benchmark does not judge readability, reusability or Lean style. The data listed no public solution for these five.

One record says the proof received no human review; correctness was checked only by the comparator.

Sources: [eval page](#) · [lean-eval](#) · [leaderboard](#) · [comparator](#)

A contribution roadmap, bottom to top

Poincaré–Bendixson

[proved by 5 AI systems on the Lean eval leaderboard; not in Mathlib](#)

Jordan curve theorem (or a plane version)

[not in Mathlib](#)

Flows and ω -limit sets

definitions in Mathlib; planar results unchecked

Global existence and maximal solutions

check current Mathlib

Local ODE existence, uniqueness, local flows

in Mathlib

Green = already there

Each layer is a sequence of small, reviewable PRs: the Mathlib-friendly path.

The [Isabelle CPP 2020 paper](#) is a ready-made example of a publication describing this kind of project.

Newton–Kantorovich and radii polynomials

Newton–Kantorovich in Lean?

- Mathlib has a polynomial Newton map only: [Mathlib.Dynamics.Newton](#).
- Could not find formalization of the Newton–Kantorovich theorem in Lean, Isabelle or Coq. (Searches can miss things.)

Nearest relative: a Lean 4 framework for the radii polynomial method (Fengyang Wang)

[arXiv:2610.02335](#)

Not a Mathlib contribution: a separate library built on Mathlib.

162

Lean files

39,691

lines of Lean

What it does

- Certifies an exact solution near a numerical approximation by checking four norm bounds and one polynomial inequality.
- Examples: polynomial zeros, Taylor initial value problems including the Lorenz system, Chebyshev approximations.
- Uses Mathlib's contraction mapping theorem, Fréchet derivatives, ODE uniqueness, lp spaces.
- For those of you who are familiar with the interval arithmetic methods for applied math, this bounding the tail in some examples.

Radii polynomials: inside and limits

How it is built

- Weighted ℓ^1 coefficient algebras for Taylor and Chebyshev series.
- Operators split into a finite block plus a diagonal tail; tail estimates reduce infinite bounds to finite checks.
- Two ways to check certificates: tactics (including [LeanCert](#)) or decidable statements over the rationals.
- The numerics (e.g. Lorenz) are computed outside Lean, in Julia; Lean checks the bounds, not the algorithm.

Shortcomings the author reports

- Mathlib gaps: a bilinear convolution API with summability, a lemma `fderiv_apply`, and smoothness of polynomial evaluation.
- Not covered: PDEs and boundary value problems; periodic orbits, invariant manifolds, connecting orbits are future work.
- Some wrappers are named as candidates for upstreaming; the paper says one specialization was later merged.

AMBER: a benchmark, not a library

Construction–Verification: A Benchmark for Formalizing Applied Mathematics in Lean 4 ([arXiv:2602.01291](https://arxiv.org/abs/2602.01291))

200

problems

4 domains, 50 problems each:
optimization, convex analysis,
numerical algebra, high-
dimensional probability

1 Construct

Define an explicit function that computes the solution. Its signature takes only the raw parameters, never the target property or its proof.



2 Verify

Prove that this specific term satisfies the required property.

- Finding: general-purpose reasoning models beat specialized theorem provers (the authors' term: “tactical overfitting”).
- Builds on Mathlib's definitions; No repository link given and nothing mentioned on upstreaming.

Links: contributing

Guides

[Contribution guidelines](#)

[How to contribute](#)

[Git guide](#)

[Style guide](#)

[Naming conventions](#)

Finding gaps

[Mathlib overview](#)

[Undergrad coverage](#)

[Missing undergrad topics](#)

[Loogle](#)

[Mathlib docs](#)

[Reservoir \(Lean package registry\)](#)

People and places

[Lean Zulip](#)

[Teams](#)

[Mathlib maintainers](#)

[Mathlib reviewers](#)

[Review dashboard \(queueboard\)](#)

Sample PRs

[#32170 gronwallBound_mono](#)

[#34709 discrete Grönwall](#)

[#26382 local flows](#)

[#20704 first attempt at local flows](#)

[#29186 IsIntegralCurve \(naming feedback\)](#)

Links: examples and alternatives

Dynamics in Mathlib

[BirkhoffSum docs](#)

[Ergodic docs](#)

[Mean ergodic theorem](#)

[\$\omega\$ -limits](#)

[Flows](#)

[Picard–Lindelöf](#)

[Newton map](#)

[Pointwise ergodic PRs: #26923, #42078, #34031](#)

Poincaré–Bendixson and Lean eval

[Isabelle/HOL, CPP 2020](#)

[Archive of Formal Proofs entry](#)

[Lean eval: Poincaré–Bendixson](#)

[Lean eval: Jordan curve](#)

[Lean eval home and leaderboard](#)

[lean-eval repository](#)

[Leaderboard data and comparator](#)

Radii polynomials and AMBER

[Radii polynomial paper \(arXiv\)](#)

[Radii polynomial HTML](#)

[Older public repo \(paper's link 404s\)](#)

[LeanCert](#)

[AMBER paper \(arXiv\)](#)

TauCeti and relatives

[TauCeti](#)

[About](#)

[Statistics](#)

[TauCeti code repo](#)

[Roadmap repo](#)

[Review repo](#)

[Lean Pool \(arXiv\)](#)

[Liquid Tensor repo](#)